
Gated Learning-Progress Exploration

Mikael Vincent Tumamos¹ Ervin Joshua Guirnela¹ Christine Peña¹

Abstract

Exploration via intrinsic rewards can accelerate learning when extrinsic feedback is sparse or deceptive, but many intrinsic objectives overemphasize novelty in high-uncertainty regions where learning is slow or unstable. We introduce *Gated Learning-Progress Exploration* (GLPE), a simple intrinsic-reward family that prioritizes transitions that both (i) induce meaningful changes in a learned feature representation and (ii) occur in regions where a forward dynamics model is actively improving. GLPE computes region-local learning progress via discrepancies between short- and long-horizon prediction-error EMAs within an online partition of feature space, and combines it with a feature-space impact term. A region-specific gate optionally suppresses intrinsic rewards in regions where prediction error remains high but learning progress is low, serving as a targeted guardrail against unproductive curiosity. We also study *GLPE (no gate)*, which uses the same base intrinsic score without any gating. We evaluate both variants with a common PPO backbone on five Gymnasium control benchmarks and compare against widely used intrinsic-reward baselines, reporting performance under both step-normalized and wall-clock-normalized views. Across the suite, GLPE (no gate) matches the strongest baseline within uncertainty, while the full gated variant is most beneficial in sparse-reward regimes where ungated intrinsic rewards can destabilize training.

1. Introduction

Deep reinforcement learning agents often learn effectively when dense extrinsic rewards provide informative gradients throughout training, but performance can degrade sharply when rewards are sparse, delayed, or shaped in ways that do not guide early exploration. Intrinsic-reward methods address this by augmenting the environment reward with an auxiliary signal that encourages exploration in the absence of external feedback (Singh et al., 2004; Barto, 2012; Oudeyer & Kaplan, 2007; Oudeyer et al., 2007). Common intrinsic objectives include novelty and visitation bonuses (Bellemare et al., 2016; Tang et al., 2016), prediction-error curiosity in learned feature spaces (Pathak et al., 2017; Burda et al., 2018a;b), and impact-based signals that encourage controllable state changes (Raileanu & Rocktäschel, 2020).

Despite strong empirical results, many intrinsic objectives exhibit a recurring failure mode: *unproductive curiosity*. Prediction error or novelty can remain high in regions that are difficult to model—because dynamics are complex, stochastic, or only weakly coupled to task progress—

causing the agent to spend substantial effort collecting experience that does not improve either the intrinsic model or the task policy (Mavor-Parker et al., 2021). This problem is especially acute in sparse-reward settings, where the intrinsic signal can dominate early learning.

A complementary perspective is to reward *learning progress*: experience is valuable when it improves a predictive model rather than merely being surprising (Schmidhuber, 1991; Oudeyer & Kaplan, 2007; Baranes & Oudeyer, 2009). Learning progress can help distinguish regions that are learnable-but-unmastered from regions that are either already predictable or effectively noisy. Practical progress-based methods must still address how to localize progress in high-dimensional observations, how to couple it to exploration-relevant change, and how to avoid regions where progress has stalled.

We propose *Gated Learning-Progress Exploration* (GLPE), a lightweight intrinsic-reward family that combines two signals computed from on-policy transitions. First, GLPE measures *impact* as the magnitude of change in a learned feature representation between consecutive observations. Second, it measures *region-local learning progress* by tracking forward-model prediction error within an online partition of the feature space using short- and long-horizon exponential moving averages. The resulting base score favors transitions that both change the representation and occur in regions where the dynamics model is improving. To

¹University of San Carlos, Cebu City, Philippines. Correspondence to: Mikael Vincent Tumamos <tumamos@mikaelvincent.dev>, Ervin Joshua Guirnela <17100409@usc.edu.ph>, Christine Peña <cf-pena@usc.edu.ph>.

mitigate unproductive curiosity, GLPE optionally applies a simple, region-specific *gate* that suppresses intrinsic shaping in regions with persistently high prediction error but low learning progress, acting as a targeted guardrail rather than a global annealing heuristic. We also study *GLPE (no gate)*, which uses the same base score without any gating and serves as a low-friction default.

Our contributions are:

- We introduce the GLPE family, combining region-local learning progress with feature-space impact under lightweight online normalization.
- We propose a simple gating mechanism that suppresses intrinsic shaping in high-error, low-progress regions and study its effect relative to an ungated variant.
- We provide an empirical study on a five-environment Gymnasium suite with a common PPO backbone, reporting both step-normalized and wall-clock-normalized comparisons, along with ablations and diagnostics of intrinsic dynamics and computational overhead.

We evaluate on MOUNTAINCAR-V0, BIPEDALWALKER-V3, and three MuJoCo locomotion tasks (ANT-V5, HALFCHIEETAH-V5, HUMANOID-V5), comparing against widely used intrinsic-reward baselines including ICM (Pathak et al., 2017), RND (Burda et al., 2018b), RIDE (Raileanu & Rocktäschel, 2020), and RIAC (Baranes & Oudeyer, 2009). Across this suite, GLPE (no gate) remains consistently competitive under both step and wall-clock views, while the gated variant is most helpful on the sparse-reward MOUNTAINCAR-V0 task, where it improves reliability and reduces instability relative to ungated shaping (Section 5).

2. Related Work

Intrinsic motivation in reinforcement learning is commonly framed as augmenting sparse extrinsic feedback with an internally generated signal that encourages exploration (Singh et al., 2004; Barto, 2012; Oudeyer & Kaplan, 2007; Oudeyer et al., 2007). Proposed intrinsic objectives span novelty and visitation bonuses (Bellemare et al., 2016; Tang et al., 2016; Ostrovski et al., 2017), information-gain and uncertainty reduction (Mohamed & Rezende, 2015; Houthoofd et al., 2016), and learned prediction-error curiosity (Stadie et al., 2015; Pathak et al., 2017; Burda et al., 2018a;b). Many modern methods apply these bonuses as reward shaping during policy optimization; unlike potential-based shaping, intrinsic rewards can change the optimal policy and are therefore evaluated empirically (Ng et al., 1999).

Prediction-error bonuses have been particularly effective

in high-dimensional settings. ICM (Pathak et al., 2017) learns a forward model in a learned feature space, while RND (Burda et al., 2018b) trains a predictor against a fixed random target. Related approaches measure novelty via episodic memory (Savinov et al., 2018), model disagreement (Pathak et al., 2019), or combinations of episodic and lifelong bonuses (Badia et al., 2020). A known limitation is that prediction error may remain high in regions that are inherently stochastic or difficult to model, leading to exploration that is intense but not task-useful; the “noisy TV” phenomenon is one concrete example (Mavor-Parker et al., 2021).

Learning-progress approaches instead reward improvements in a predictive model over time, encouraging experience that is learnable but not yet mastered (Schmidhuber, 1991; Oudeyer & Kaplan, 2007). In continuous control, RIAC (Baranes & Oudeyer, 2009) localizes progress by maintaining an adaptive partition of state space and using progress estimates to guide exploration. The GLPE family builds on this lineage by (i) computing region-local learning progress in a learned latent space rather than in hand-chosen state coordinates, (ii) combining progress with a feature-space impact term related to controllable change (Raileanu & Rocktäschel, 2020), and (iii) introducing an explicit gate that suppresses intrinsic shaping when prediction error stays high but progress stalls. This gate serves as a lightweight guardrail against unproductive curiosity rather than an alternative intrinsic objective.

3. Method

3.1. Problem setup

We consider an episodic Markov decision process with observations $o_t \in \mathcal{O}$, actions $a_t \in \mathcal{A}$, and extrinsic rewards r_t^{ext} . A stochastic policy $\pi_\theta(a \mid o)$ is trained to maximize the expected discounted return.

We augment the extrinsic reward with an intrinsic term computed from single-step transitions,

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_t^{\text{int}}, -r_{\max}, r_{\max}), \quad (1)$$

where $\eta_t \geq 0$ controls the intrinsic–extrinsic trade-off and $r_{\max} > 0$ bounds the magnitude of intrinsic shaping. For environment-terminal transitions we set $r_t^{\text{int}} = 0$ so that intrinsic shaping does not depend on episode termination.

We propose two intrinsic reward formulations within a shared framework: GLPE and GLPE (no gate). Both combine feature-space impact with region-local learning progress. GLPE additionally introduces a region-specific gate that suppresses intrinsically rewarding but unproductive experience. We refer to these two variants collectively as the *GLPE family*.

3.2. Latent dynamics model

Let ϕ_ω be a learned encoder producing a d -dimensional feature vector $z_t = \phi_\omega(o_t)$. We train (i) a forward dynamics predictor f_ψ that maps (z_t, a_t) to a prediction of the next feature vector, and (ii) an inverse dynamics predictor g_ξ that maps (z_t, z_{t+1}) to an action distribution. The inverse model is instantiated as a classifier for discrete actions and as a Gaussian likelihood model for continuous actions.

Training uses the combined loss

$$\mathcal{L}_{\text{dyn}}(t) = \beta_{\text{fwd}} \mathcal{L}_{\text{fwd}}(t) + \beta_{\text{inv}} \mathcal{L}_{\text{inv}}(t), \quad (2)$$

with weights $\beta_{\text{fwd}}, \beta_{\text{inv}} > 0$. The forward loss is the mean-squared error in feature space,

$$\mathcal{L}_{\text{fwd}}(t) = \frac{1}{d} \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2, \quad (3)$$

and we define the per-transition prediction error

$$e_t = \mathcal{L}_{\text{fwd}}(t). \quad (4)$$

The intrinsic rewards below depend on (z_t, z_{t+1}) and e_t , but do not require access to privileged environment state.

3.3. Online region partitioning in latent space

To localize learning progress, we maintain an online partition of the latent space into regions. Each region corresponds to a leaf in a binary space-partitioning tree over the d -dimensional feature space.

Given an embedding z_t , we assign it to a leaf region ρ_t by routing it through the tree. Each leaf maintains a visit counter and, when it can still be split, a buffer of stored points used to choose a split dimension and threshold. When the number of assigned points to a leaf reaches a capacity C and the leaf depth is below a maximum depth D_{max} , the leaf is split. We choose the highest-variance coordinate that yields a non-degenerate split and split at the median value along that coordinate. When a leaf splits, one child retains the parent region identifier (and its accumulated statistics), while the other child is assigned a new identifier whose statistics are created on its first visit. This procedure yields a growing set of regions that adaptively refines frequently visited parts of feature space.

Region statistics (defined next) are updated using the region assignment based on the current embedding z_t . The capacity C and maximum depth D_{max} control the granularity of the partition.

3.4. Region-local learning progress

For each region r we maintain two exponential moving averages (EMAs) of the forward prediction error: a long-horizon average μ_r^{long} and a short-horizon average μ_r^{short} .

When a transition at time t is assigned to region $\rho_t = r$, we update

$$\mu_r^{\text{long}} \leftarrow \beta_{\text{long}} \mu_r^{\text{long}} + (1 - \beta_{\text{long}}) e_t, \quad (5)$$

$$\mu_r^{\text{short}} \leftarrow \beta_{\text{short}} \mu_r^{\text{short}} + (1 - \beta_{\text{short}}) e_t, \quad (6)$$

with $0 < \beta_{\text{long}} < 1$, $0 < \beta_{\text{short}} < 1$, and typically $\beta_{\text{long}} > \beta_{\text{short}}$ so that μ_r^{long} changes more slowly. On the first visit to a newly created region identifier, we initialize $\mu_r^{\text{long}} = \mu_r^{\text{short}} = e_t$.

We define the current learning progress of a region r as the positive part of the discrepancy between the long and short averages,

$$\text{LP}(r) = \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}}). \quad (7)$$

For each transition we use the region assignment ρ_t and set $\text{LP}_t = \text{LP}(\rho_t)$. This quantity is large when the short-term error has recently dropped below the longer-term baseline, indicating active model improvement in that region.

3.5. Impact in feature space

We also compute a feature-space impact term that encourages transitions that change the learned representation,

$$I_t = \|z_{t+1} - z_t\|_2. \quad (8)$$

Unlike count-based novelty, impact depends on the learned representation and can capture structured changes in high-dimensional observations.

3.6. Component-wise scale stabilization

The magnitudes of I_t and LP_t can vary across tasks and throughout training. To obtain a stable scale without task-specific tuning, we maintain running root-mean-square (RMS) normalizers for each component using an exponential average of squared values. For a scalar signal x_t we update an accumulator

$$v \leftarrow \beta_{\text{rms}} v + (1 - \beta_{\text{rms}}) x_t^2, \quad (9)$$

and normalize by $\text{RMS}(x_t) = \sqrt{v + \varepsilon}$ with a small $\varepsilon > 0$.

Applying this independently to I_t and LP_t yields normalized components

$$\tilde{I}_t = \frac{I_t}{\text{RMS}(I_t)}, \quad \tilde{\text{LP}}_t = \frac{\text{LP}_t}{\text{RMS}(\text{LP}_t)}. \quad (10)$$

We then define a shared base intrinsic score

$$u_t = \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{\text{LP}}_t, \quad (11)$$

with nonnegative weights $\alpha_{\text{impact}}, \alpha_{\text{LP}}$. Both proposed methods use the same u_t and differ only in whether it is gated.

3.7. GLPE (no gate)

GLPE (no gate) is a region-aware intrinsic reward that combines feature-space impact and region-local learning progress without any explicit suppression mechanism. The intrinsic reward is

$$r_t^{\text{int}} = u_t. \quad (12)$$

This variant preserves the locality and nonstationarity tracking provided by the region partition and EMAs, while avoiding additional gating hyperparameters.

3.8. GLPE: gated learning-progress exploration

GLPE augments the same base score u_t with a binary, region-specific gate that suppresses intrinsic rewards in regions that appear difficult to predict yet show little or no learning progress.

Global robust thresholds. At any time, let $\mathcal{R}$ denote the set of regions that have been visited at least once, and let $\text{LP}(r)$ denote the current learning progress in region r . We compute robust global reference values using medians across visited regions,

$$\begin{aligned} \text{LP}_{\text{med}} &= \text{median}_{r \in \mathcal{R}} \text{LP}(r), \\ e_{\text{med}} &= \text{median}_{r \in \mathcal{R}} \mu_r^{\text{short}}. \end{aligned} \quad (13)$$

We define a learning-progress threshold $\tau_{\text{LP}} = \kappa \text{LP}_{\text{med}}$ with multiplier $\kappa > 0$, and a scale-free normalized error score

$$s_r = \frac{\mu_r^{\text{short}}}{e_{\text{med}} + \varepsilon}, \quad (14)$$

where $\varepsilon > 0$ avoids division by zero. A region r is considered *unproductive* at a visit if it simultaneously has low learning progress and high normalized error,

$$\text{LP}(r) < \tau_{\text{LP}} \quad \text{and} \quad s_r > \tau_s, \quad (15)$$

with $\tau_s > 0$ a fixed threshold.

Gate dynamics. Each region maintains a gate value $g_r \in \{0, 1\}$ and two small counters tracking consecutive unproductive and productive visits. Gating is only activated once at least R_{min} regions have been visited so that the medians are meaningful. When gating is active, the update uses persistence and hysteresis: (i) if $g_r = 1$ and the region is unproductive for K consecutive visits, set $g_r \leftarrow 0$; and (ii) if $g_r = 0$, re-enable the region when $\text{LP}(r) > h \tau_{\text{LP}}$ for two consecutive visits, where $h > 1$ is a hysteresis multiplier. If gating is inactive (fewer than R_{min} visited regions), we set $g_r \leftarrow 1$ and reset the counters.

The intrinsic reward for GLPE is then

$$r_t^{\text{int}} = g_{\rho_t} u_t. \quad (16)$$

3.9. On-policy optimization and intrinsic scheduling

We optimize the policy with PPO using the augmented reward r_t . Training details are given in Section 4.

For the GLPE family, we anneal the strength of intrinsic shaping over the course of training using a cosine taper. Let $p \in [0, 1]$ denote training progress as a fraction of total environment steps. Given a start fraction p_{start} and end fraction p_{end} with $p_{\text{start}} < p_{\text{end}}$, we define

$$w(p) = \begin{cases} 1, & p \leq p_{\text{start}}, \\ \frac{1}{2} \left(1 + \cos\left(\pi \frac{p - p_{\text{start}}}{p_{\text{end}} - p_{\text{start}}}\right) \right), & p_{\text{start}} < p < p_{\text{end}}, \\ 0, & p \geq p_{\text{end}}, \end{cases} \quad (17)$$

and set $\eta_t = \eta w(p_t)$ in the augmented reward. When p_{start} and p_{end} are omitted, we hold $\eta_t = \eta$ fixed throughout training.

The latent dynamics model parameters (ω, ψ, ξ) are updated using the same on-policy transitions that generate intrinsic rewards, while the region statistics, gates, and RMS normalizers are updated online during intrinsic computation.

4. Experimental Setup

4.1. Tasks and environments

We evaluate exploration and control on five standard Gymnasium benchmarks spanning discrete and continuous control: MOUNTAINCAR-V0, BIPEDALWALKER-V3, and the MuJoCo locomotion tasks ANT-V5, HALF-CHEETAH-V5, and HUMANOID-V5. Across all experiments we use the default environment reward functions and termination conditions, with no domain randomization and no additional frame skipping. Training uses vectorized environments with B parallel instances and rollouts of length T per instance. We set (B, T) so that the nominal on-policy batch size per PPO update is

$$N = B \times T = 16,384 \quad (18)$$

transitions, varying (B, T) only to accommodate environment throughput and episode structure. To match the total step budget exactly, the final update may use a smaller batch when fewer than N steps remain. Table 1 summarizes the suite, interaction budgets, and parallelization settings. For each environment we evaluate every method using the same set of training seeds (Table 1) and enable deterministic execution where supported.

Training budgets beyond the early-learning regime.

The total environment-step budgets in Table 1 are intentionally generous so that runs extend beyond the initial improvement phase. This provides headroom to assess long-horizon stability of exploration behavior and intrinsic shaping, including late-training drift, regressions, or oscillations.

```

1: Input: on-policy transitions  $\{(o_t, a_t, o_{t+1})\}_{t=1}^N$ ; encoder  $\phi_\omega$ ; forward model  $f_\psi$ ; inverse model  $g_\xi$ ; region assignment function  $\rho$ ;
   region EMAs  $\mu_r^{\text{short}}, \mu_r^{\text{long}}$ ; RMS accumulators; gate states  $g_r$  (optional)
2: Output: intrinsic rewards  $\{r_t^{\text{int}}\}_{t=1}^N$ ; updated region statistics; updated intrinsic model parameters
3: for  $t = 1$  to  $N$  do
4:    $z_t \leftarrow \phi_\omega(o_t), z_{t+1} \leftarrow \phi_\omega(o_{t+1})$ 
5:    $e_t \leftarrow \frac{1}{d} \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2$ 
6:   Assign region  $r \leftarrow \rho(z_t)$  (inserting  $z_t$  and splitting leaves when needed)
7:   Update region EMAs  $\mu_r^{\text{short}}, \mu_r^{\text{long}}$  with  $e_t$ 
8:    $\text{LP}_t \leftarrow \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}})$ 
9:    $I_t \leftarrow \|z_{t+1} - z_t\|_2$ 
10:  Normalize  $I_t$  and  $\text{LP}_t$  with running RMS to get  $\tilde{I}_t, \tilde{\text{LP}}_t$ 
11:   $u_t \leftarrow \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{\text{LP}}_t$ 
12:  if gating enabled then
13:    Compute robust global medians  $\text{LP}_{\text{med}}, e_{\text{med}}$  across visited regions
14:    Update gate  $g_r$  using persistence and hysteresis rules
15:     $r_t^{\text{int}} \leftarrow g_r u_t$ 
16:  else
17:     $r_t^{\text{int}} \leftarrow u_t$ 
18:  end if
19: end for
20: Update intrinsic model parameters  $(\omega, \psi, \xi)$  using  $\{(z_t, a_t, z_{t+1})\}$  and supervised losses

```

Figure 1. High-level pseudocode for computing GLPE-family intrinsic rewards within one PPO update.

Environment	B	T	N	Total steps	Seeds
MOUNTAINCAR-V0	16	1,024	16,384	3,000,000	10
BIPEDEALWALKER-V3	8	2,048	16,384	7,000,000	8
ANT-V5	8	2,048	16,384	15,000,000	8
HALFCHEETAH-V5	8	2,048	16,384	15,000,000	8
HUMANOID-V5	4	4,096	16,384	30,000,000	5

Table 1. Benchmark suite and training budgets. B is the number of parallel environment instances, T is the rollout horizon per instance, and $N = B \times T$ is the number of transitions used in each PPO update. For every run we evaluate saved checkpoints offline using 20 episodes (Section 4.4).

4.2. Methods compared

We compare two proposed intrinsic-reward methods, GLPE and GLPE (no gate) (Section 3), against a set of commonly used exploration baselines: Vanilla PPO, ICM, RND, RIDE, and RIAC. Vanilla PPO optimizes only the extrinsic environment reward. ICM uses forward-model prediction error in a learned feature space as an intrinsic signal, while RND uses the prediction error of a trainable predictor against a fixed random target network. RIDE rewards feature-space impact, down-weighted by episodic visitation counts obtained by discretizing features, and RIAC uses region-local learning progress computed from changes in forward-model error within an adaptive partition of feature space. All methods use the same PPO backbone and network architectures. Within each environment we keep PPO hyperparameters fixed across methods to isolate the effect of the intrinsic objective.

For intrinsic-reward methods, the policy is trained with the augmented reward defined in Section 3, combining the environment reward with a scaled and clipped intrinsic term. We use the same intrinsic scaling coefficient η and clipping range r_{max} for all intrinsic methods within a given environ-

ment (Table 2), so differences arise from the intrinsic signal itself rather than from reward-scale tuning. GLPE (no gate) uses the same base intrinsic score as GLPE but disables the gating mechanism.

4.3. RL backbone and training protocol

All agents are trained with Proximal Policy Optimization (PPO; (Schulman et al., 2017)) with generalized advantage estimation (GAE; (Schulman et al., 2015)). The policy and value function are parameterized by separate multilayer perceptrons with two hidden layers of width 256 and ReLU activations. For continuous-control tasks, the policy outputs a diagonal Gaussian distribution. Actions are squashed to the environment bounds when those bounds are finite.

Each training iteration proceeds as follows: (i) collect up to $N = 16,384$ on-policy transitions by running the current policy in B vectorized environments for T steps; (ii) compute the intrinsic reward for each transition (when applicable), set intrinsic rewards to zero on environment-terminal transitions and on time-limit truncations without a final observation, and form the total reward used for advantage estimation; (iii) compute advantages and value targets with

Environment	η	$r_{\max}$	α_{LP}	p_{start}	p_{end}	C	$D_{\max}$	β_{long}	β_{short}	κ	τ_s	K	$R_{\min}$
MOUNTAINCAR-V0	0.05	4.0	0.5	0.05	0.80	128	10	0.995	0.90	0.010	2.0	5	8
BIPEDALWALKER-V3	0.08	4.0	0.5	0.12	0.75	200	12	0.995	0.90	0.010	2.0	5	16
ANT-V5	0.06	4.0	0.6	0.10	0.70	256	12	0.997	0.92	0.012	2.5	8	64
HALFCHEETAH-V5	0.04	5.0	0.5	0.05	0.75	256	12	0.995	0.90	0.010	2.0	5	32
HUMANOID-V5	0.06	5.0	0.5	0.05	0.80	256	12	0.998	0.92	0.010	2.0	5	32

Table 2. GLPE hyperparameters by environment. We fix $\alpha_{\text{impact}} = 1.0$ for all tasks. p_{start} and p_{end} define the cosine taper schedule for η_t as a function of training progress p (Section 3); this taper is used for GLPE and GLPE (no gate). C and $D_{\max}$ are the region capacity and maximum depth for the online latent-space partition. κ sets the learning-progress threshold via $\tau_{\text{LP}} = \kappa \text{LP}_{\text{med}}$, τ_s is the normalized-error threshold, K is the number of consecutive unproductive visits required to gate off a region, and $R_{\min}$ is the minimum number of visited regions required before gating is activated (Section 3). The hysteresis multiplier is fixed to $h = 2.0$ for all tasks. GLPE (no gate) uses the same settings but does not apply gating.

GAE, bootstrapping across time-limit truncations when a valid final observation is available and treating truncations without a final observation as terminal for bootstrapping purposes; and (iv) perform multiple epochs of PPO updates over shuffled minibatches of the collected batch. We use Adam optimizers for both the policy and value networks, normalize advantages within each update batch, and clip gradient norms to 1.0.

We normalize vector observations using a running mean and variance estimated online from collected rollouts. We apply the same normalization to the inputs of the policy, value, and intrinsic modules.

Intrinsic reward computation and updates. Trainable intrinsic models are updated once per PPO iteration using the same on-policy batch used for PPO. For methods whose intrinsic output is not internally normalized (e.g., ICM, RND, and RIDE), we apply a running RMS normalization to the raw intrinsic signal before scaling and clipping. For methods that produce a normalized intrinsic output by construction (RIAC and the GLPE family), we directly apply scaling and clipping. For GLPE and GLPE (no gate), we additionally anneal the intrinsic coefficient η_t using the cosine taper described in Section 3, with task-specific start and end fractions listed in Table 2.

4.4. Evaluation protocol

We evaluate performance using undiscounted episodic return from the environment (extrinsic reward only). We do not run evaluation rollouts online during training; instead, we evaluate saved checkpoints offline. Each evaluated checkpoint is run for 20 episodes in a separate environment instance using deterministic action selection (distribution mode). Episodes are reset with fixed per-episode seeds derived from the training seed. For each environment and method, we run multiple independent training seeds (Table 1) and aggregate results across seeds.

Checkpointing and logging. During training we checkpoint the full agent state at step 0, at nine additional warmup

points approximately evenly spaced within the first checkpoint interval, and then at a fixed checkpoint interval thereafter (set to 10% of the total step budget per environment), with a final checkpoint at the end of training. We additionally log scalar metrics every fixed number of environment steps (set to 2% of the total step budget).

4.5. Efficiency measurements

In addition to sample efficiency (learning as a function of environment steps), we track computational efficiency using wall-clock time. All timings are measured in the same execution setting used for training (CPU in our experiments) and therefore reflect end-to-end optimization overhead. For each PPO iteration we log per-component wall-clock time spent in environment interaction, policy inference, intrinsic computation, intrinsic-module updates, advantage computation, and PPO optimization. We use these logs to form cumulative training time.

When reporting scalar summaries of learning curves, we compute two AUC metrics via trapezoidal integration of mean return: step-AUC (return versus environment steps, divided by the environment’s step budget) and wall-clock AUC (return versus cumulative training time). For wall-clock AUC, we use a *common time budget* per environment defined as the minimum final wall-clock time attained among the compared methods. This ensures that summaries do not rely on extrapolation beyond the measured runtime of any method.

4.6. Implementation details and hyperparameters

Tables 3 and 2 list the PPO and GLPE hyperparameters used in all experiments. Unless specified in the tables, we use $\gamma = 0.99$ and value-loss coefficient 0.5 for PPO.

For methods that learn a latent dynamics model (ICM, RIDE, RIAC, and the GLPE family), the shared encoder produces 128-dimensional features and is implemented as a two-layer MLP (256 units per layer, ReLU) for these vector-observation tasks. Forward and inverse model losses are weighted equally. The model is trained with Adam at learn-

ing rate 3×10^{-4} , and intrinsic-model gradient norms are clipped to 5.0.

5. Results and Discussion

We compare GLPE and GLPE (no gate) against Vanilla PPO and four intrinsic-reward baselines (ICM, RND, RIDE, RIAC) on the five-task suite from Section 4. Unless otherwise stated, we report *extrinsic* evaluation return under the deterministic offline evaluation protocol (Section 4.4).

5.1. Learning curves and final performance

Figure 2 shows evaluation learning curves versus environment steps. Across tasks, GLPE (no gate) tracks the best-performing baseline closely and exhibits stable between-seed behavior. The gated variant differs mainly in sparse-reward or exploration-sensitive regimes: it is most beneficial on MOUNTAINCAR-V0 and can be overly conservative on HUMANOID-V5, where variance across seeds dominates.

Table 4 summarizes final-checkpoint return, while Table 5 reports step-AUC, which rewards both early learning and sustained performance over the fixed interaction budget.

On MOUNTAINCAR-V0, GLPE attains the best final return and the strongest step-AUC among all compared methods, and both GLPE variants solve in all seeds (Tables 4 and 6). On BIPEDALWALKER-V3, GLPE (no gate) improves rapidly and remains competitive by step-AUC, but many runs plateau just below the solved threshold of 300, making solved counts sensitive to small differences near the cutoff (Table 6). On the MuJoCo locomotion tasks, where extrinsic reward is comparatively dense, the two GLPE variants behave similarly and remain within a modest gap of the strongest intrinsic baseline under both summaries (Tables 4 and 5). HUMANOID-V5 exhibits particularly high variance, with wide confidence intervals across all methods, so method rankings are not stable under bootstrap uncertainty.

Some diagnostic plots additionally include auxiliary GLPE variants (e.g., cached medians and single-component ablations) to isolate computational and signal contributions; we treat these as supporting analyses rather than primary baselines.

5.2. Ablations: impact vs. learning progress

To probe which components drive performance in different regimes, Table 7 reports final returns for GLPE and two single-term ablations on representative tasks.

On sparse-reward MOUNTAINCAR-V0, combining impact and learning progress is critical: each single-term variant falls substantially short of full GLPE. In contrast, on dense-reward ANT-V5 the impact-only variant is competitive, and

on near-threshold BIPEDALWALKER-V3 the LP-only variant comes closest to the solved cutoff. These patterns motivate the combined score as a robust default when the “right” intrinsic driver is task-dependent.

5.3. Thresholded reliability and steps-to-threshold

Learning curves can obscure whether a method is merely slower or is failing entirely. We therefore evaluate reliability at fixed return thresholds using steps-to-threshold statistics (Figure 3) and success counts (Tables 8 and 6). We use task-specific solved thresholds (MOUNTAINCAR-V0: -110, BIPEDALWALKER-V3: 300, ANT-V5: 5,000, HALFCHEETAH-V5: 6,000, HUMANOID-V5: 3,000) and additionally report results at 25% and 50% of these levels.

At the 50% threshold, GLPE (no gate) reaches the target in all seeds on four tasks and in 4/5 seeds on HUMANOID-V5, matching the best baseline reach count on every environment (Table 8). The gap between 50% and 100% success is concentrated in high-variance or near-cutoff regimes: HUMANOID-V5 and BIPEDALWALKER-V3 show many runs that reach competent behavior without consistently clearing the full solved cutoff (Table 6). On MOUNTAINCAR-V0, by contrast, both GLPE variants reach the solved threshold quickly and reliably, and several baselines exhibit occasional failures that remain near the minimum return (Figure 2).

5.4. Intrinsic shaping dynamics and gating behavior

The GLPE family is designed as a transient exploration aid rather than a permanent auxiliary objective. Figure 4 visualizes the cosine taper schedule used to anneal the applied intrinsic reward late in training, and Figure 5 decomposes the training-time rollout reward into extrinsic and intrinsic components. Compared to baselines that maintain a nonzero intrinsic term throughout training, GLPE and GLPE (no gate) smoothly reduce the intrinsic contribution while the underlying (untapered) intrinsic signal remains present.

To make the gate’s effect concrete, Figure 6 shows state-space projections of final-checkpoint trajectories with timesteps colored by whether intrinsic shaping is active. Across environments, gate-off points occupy a small and typically localized subset of the visited distribution. This supports the interpretation of gating as a targeted guardrail: it suppresses shaping only in narrow regions that appear persistently high-error and low-progress, rather than globally weakening exploration.

5.5. Wall-clock efficiency and computational overhead

Intrinsic rewards can change not only sample efficiency but also end-to-end training cost. Figure 7 reports wall-clock AUC of evaluation return under a common per-environment

Environment	LR	Epochs	Minib.	Clip	λ	Ent.	v -clip	KL stop
MOUNTAINCAR-V0	3.0×10^{-4}	10	16	0.20	0.95	0.00	0.00	0.06
BIPEDALWALKER-V3	5.0×10^{-4}	5	16	0.25	0.95	0.00	0.00	0.06
ANT-V5	1.5×10^{-4}	15	64	0.20	0.95	0.00	0.20	0.04
HALFCHEETAH-V5	3.0×10^{-4}	10	32	0.20	0.95	0.01	0.20	0.03
HUMANOID-V5	2.0×10^{-4}	5	32	0.20	0.97	0.01	0.20	0.03

Table 3. PPO hyperparameters by environment. “Minib.” is the number of minibatches per update (each epoch iterates once over all N samples). “Ent.” is the entropy regularization coefficient. “ v -clip” is the value-function clipping range (0 disables value clipping). “KL stop” is the approximate KL threshold for early stopping within an update.

Environment	GLPE	GLPE (no gate)	Best baseline
MOUNTAINCAR-V0	-96.2 [-96.5, -95.9]	-97.1 [-99.0, -96.0]	RIAC: -106.6 [-127.5, -95.9]
BIPEDALWALKER-V3	266.5 [214.0, 301.3]	294.4 [285.9, 303.7]	Vanilla PPO: 302.2 [291.2, 309.4]
ANT-V5	4,961 [4,421, 5,359]	4,961 [4,436, 5,349]	RIAC: 5,384 [5,118, 5,625]
HALFCHEETAH-V5	6,889 [6,565, 7,285]	6,908 [6,548, 7,280]	RIDE: 7,106 [6,901, 7,299]
HUMANOID-V5	1,970 [546, 4,355]	2,858 [1,428, 4,454]	ICM: 2,874 [770, 4,956]

Table 4. Mean extrinsic return at the final checkpoint (latest by environment steps). Values are means over seeds; each seed is evaluated for 20 deterministic (mode) episodes, and confidence intervals are 95% bootstrap intervals over seeds. Higher is better; for MOUNTAINCAR-V0 this corresponds to less negative return. The best baseline is selected (per environment) among Vanilla PPO, ICM, RND, RIDE, and RIAC by highest mean return.

time horizon (Section 4.5), ensuring that slower methods are not advantaged by extrapolation beyond measured runtime. Across BIPEDALWALKER-V3, HALFCHEETAH-V5, ANT-V5, and HUMANOID-V5, GLPE (no gate) remains close to the strongest baseline under this wall-clock view. Full GLPE is typically further back, consistent with the additional computation required by gating and robust statistics.

Figure 8 decomposes per-update runtime into environment interaction, PPO optimization, and intrinsic components. On the more expensive MuJoCo tasks, environment stepping and PPO optimization dominate total time, making intrinsic overhead a secondary factor; on MOUNTAINCAR-V0, where environment interaction is cheap, intrinsic overhead plays a larger role in wall-clock comparisons.

Table 9 isolates one identifiable cost of the gated variant—recomputing global medians for robust thresholds—and shows that caching these statistics can substantially increase throughput in a standalone microbenchmark. Because cached medians are slightly stale and can change gating decisions, we treat caching as an optional engineering knob rather than part of the core GLPE comparison.

6. Conclusion

Intrinsic-reward shaping can substantially accelerate learning when extrinsic feedback is sparse or delayed, but naive novelty or prediction-error bonuses can also concentrate exploration in regions where progress is slow, unstable, or effectively noisy. We introduced the *GLPE family*, which ties intrinsic reward to two complementary signals computed from on-policy transitions: feature-space impact and region-local learning progress estimated from short- and

long-horizon prediction-error averages in an online latent-space partition. The full GLPE variant adds a simple region-wise gate that suppresses intrinsic shaping when prediction error stays high while learning progress stalls, providing a targeted guardrail against unproductive curiosity; GLPE (no gate) uses the same base score without this additional thresholding.

Across five Gymnasium control benchmarks and a shared PPO backbone, GLPE (no gate) is the most consistent default, remaining competitive with strong intrinsic baselines under both step-normalized and wall-clock-normalized views. The gated variant is most valuable in sparse-reward regimes such as MOUNTAINCAR-V0, where it improves reliability and reduces instability relative to ungated shaping, but it can be unnecessarily conservative in high-variance settings such as HUMANOID-V5. Finally, the cosine taper schedule makes intrinsic shaping intentionally transient, smoothly reducing its contribution late in training so that final performance reflects primarily the learned task policy.

This study is limited to vector-observation benchmarks and on-policy PPO, and (as with most intrinsic bonuses) the shaping signal we apply is not policy-invariant. Future work can extend GLPE to richer observation modalities and alternative training regimes, and can make the gated variant cheaper and more adaptive without changing the underlying intrinsic objective.

Environment	GLPE	GLPE (no gate)	Best baseline
MOUNTAINCAR-V0	-107.0 [-114.8, -101.8]	-111.3 [-128.7, -101.4]	RIDE: -113.7 [-134.2, -101.9]
BIPEDALWALKER-V3	249.0 [223.9, 270.7]	255.2 [230.0, 275.6]	ICM: 257.3 [229.4, 278.3]
ANT-V5	3,402 [3,067, 3,708]	3,402 [3,067, 3,707]	RND: 3,565 [3,202, 3,876]
HALFCHEETAH-V5	5,005 [4,708, 5,316]	4,998 [4,700, 5,296]	RIDE: 5,208 [5,019, 5,410]
HUMANOID-V5	1,583 [551, 3,360]	1,665 [769, 2,921]	ICM: 1,781 [829, 2,797]

Table 5. Step-AUC of deterministic evaluation return versus cumulative environment steps, computed by trapezoidal integration of the mean return-by-steps curve over evaluation checkpoints and dividing by the environment’s step budget. This yields the average evaluation return over training, rewarding methods that learn quickly and sustain high returns. Intervals are obtained by integrating the per-checkpoint 95% bootstrap confidence bands over seeds and should be interpreted as an approximate uncertainty range for the integrated metric. Higher is better; for MOUNTAINCAR-V0 this corresponds to less negative return. The best baseline is selected (per environment) among Vanilla PPO, ICM, RND, RIDE, and RIAC by highest mean step-AUC.

Environment	GLPE	GLPE (no gate)	Most reliable baseline
MOUNTAINCAR-V0	10/10; 0.35 M	10/10; 0.35 M	RIDE: 9/10; 0.44 M
BIPEDALWALKER-V3	4/8; 3.10 M	2/8; 3.04 M	RIAC: 8/8; 4.09 M
ANT-V5	7/8; 10.39 M	7/8; 10.39 M	RIAC: 8/8; 10.22 M
HALFCHEETAH-V5	8/8; 9.68 M	8/8; 9.42 M	RIDE: 8/8; 9.03 M
HUMANOID-V5	1/5; 6.73 M	2/5; 18.23 M	ICM: 2/5; 13.93 M

Table 6. Reliability and speed at the solved threshold. Each cell reports the number of seeds that reached the task’s solved threshold by the end of training and the median steps (in millions) to first reach it, computed by linearly interpolating between deterministic evaluation checkpoints (Figure 3). Medians are taken over successful seeds only. The “most reliable baseline” is the method with the highest solve count among Vanilla PPO, ICM, RND, RIDE, and RIAC (ties broken by lower median steps).

Environment	GLPE	Impact-only	LP-only
MOUNTAINCAR-V0	-96.2 [-96.5, -95.9]	-111.8 [-137.0, -96.2]	-106.6 [-127.4, -96.0]
BIPEDALWALKER-V3	266.5 [214.0, 301.3]	277.5 [256.0, 296.5]	298.8 [285.5, 308.1]
ANT-V5	4,961 [4,421, 5,359]	5,233 [4,544, 5,663]	4,599 [3,617, 5,401]

Table 7. Final-checkpoint mean extrinsic return for GLPE and component ablations. Impact-only sets $\alpha_{LP}=0$ and LP-only sets $\alpha_{impact}=0$. Brackets show 95% bootstrap confidence intervals over training seeds. Higher is better (less negative is better on MOUNTAINCAR-V0).

Environment	GLPE	GLPE (no gate)	Most reliable baseline
MOUNTAINCAR-V0	10/10; 0.12 M	10/10; 0.11 M	RIDE: 9/10; 0.13 M
BIPEDALWALKER-V3	8/8; 0.44 M	8/8; 0.41 M	ICM: 8/8; 0.41 M
ANT-V5	8/8; 4.61 M	8/8; 4.61 M	RND: 8/8; 4.18 M
HALFCHEETAH-V5	8/8; 2.01 M	8/8; 2.01 M	Vanilla PPO: 8/8; 1.28 M
HUMANOID-V5	2/5; 14.51 M	4/5; 11.47 M	ICM: 4/5; 10.32 M

Table 8. Reliability and speed at 50% of the solved threshold. Each cell reports the number of seeds that reached 50% of the task’s solved threshold by the end of training and the median steps (in millions) to first reach that level, computed by linearly interpolating between deterministic evaluation checkpoints (Figure 3). For tasks with negative solved thresholds (MOUNTAINCAR-V0), the 50% level uses the same monotonic mapping as in Figure 3. Medians are taken over successful seeds only. The “most reliable baseline” is the method with the highest reach count among Vanilla PPO, ICM, RND, RIDE, and RIAC (ties broken by lower median steps).

Setting	Throughput (transitions/s)	Speedup
Cache off ($k=1$)	20,877	1.00×
Cache on ($k=64$)	100,173	4.83×

Table 9. Microbenchmark for caching the global medians used by GLPE’s gate. We measure median throughput over 9 trials for the same workload when recomputing global medians every update (cache off) versus recomputing every $k=64$ updates and reusing cached values in between (cache on). The cache increases throughput by 4.83× at the cost of slightly stale medians, which can change gating decisions when $k > 1$.

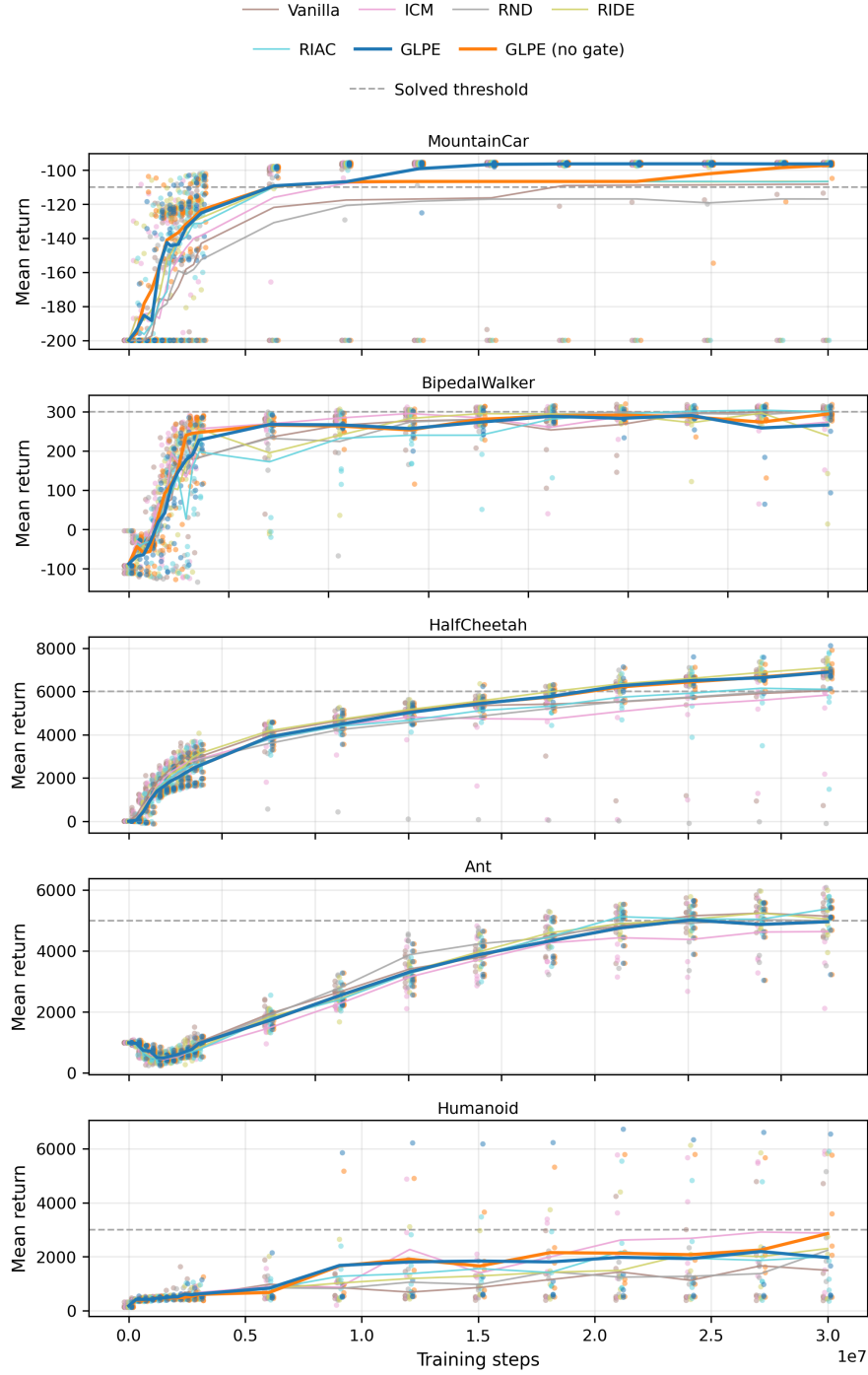


Figure 2. Evaluation learning curves for GLPE and baseline intrinsic-reward methods. Each panel plots mean *extrinsic* return (higher is better) versus environment steps for one task. Solid lines show the mean across seeds at each checkpoint; markers show the corresponding per-seed means (20 evaluation episodes per checkpoint). The horizontal dashed line is a fixed “solved” return threshold used for summary analyses. These curves summarize both learning speed and cross-seed stability; in particular, tight clustering of per-seed markers indicates robust performance.

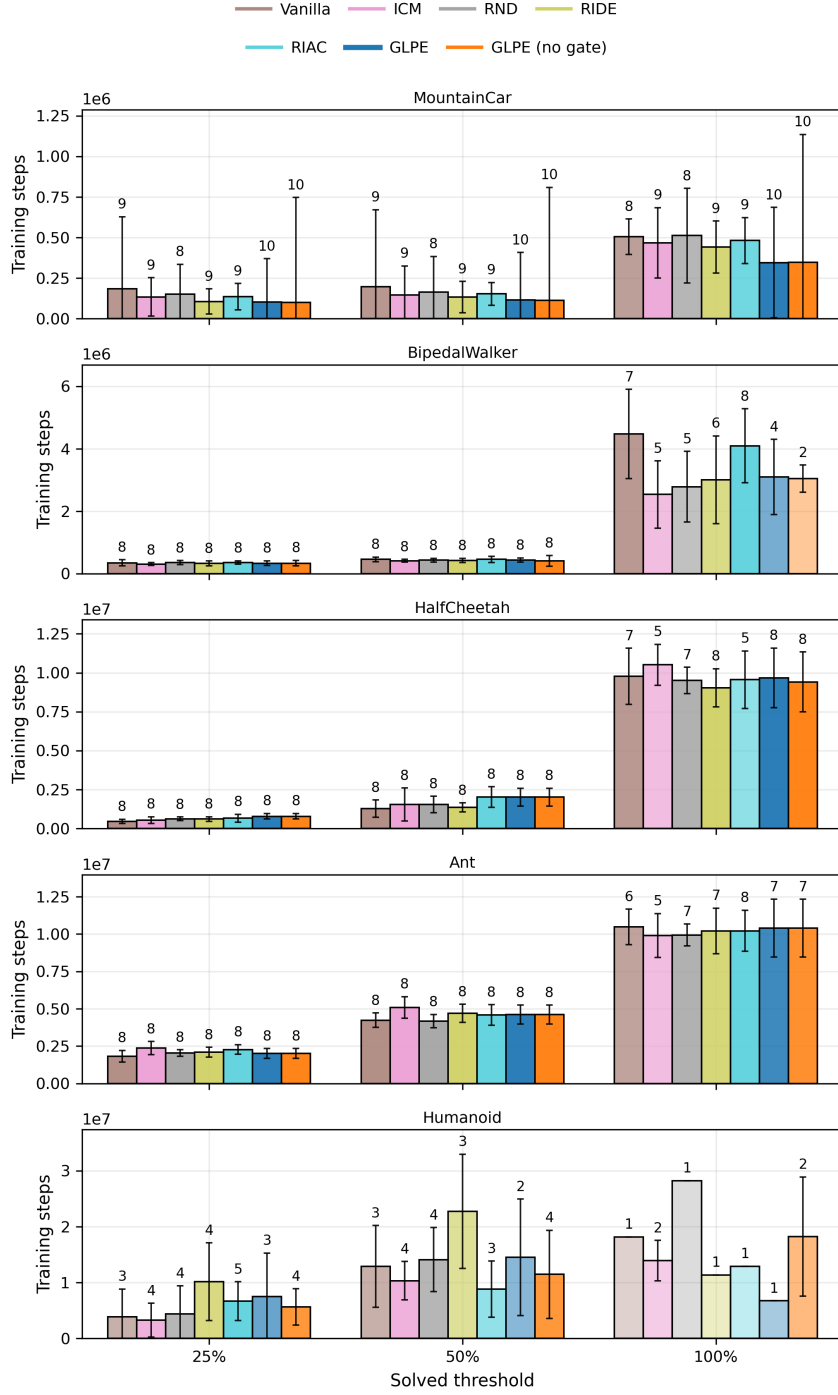


Figure 3. Sample efficiency and reliability at fixed return thresholds. For each task, bars report the median number of environment steps required to first reach a target return level, computed by linearly interpolating between evaluation checkpoints. The three x-axis positions correspond to 25%, 50%, and 100% of that task’s solved threshold (the dashed line in Figure 2); for tasks with negative solved thresholds (MountainCar), the intermediate thresholds are defined by the same monotonic mapping used in the plotting code. Error bars show the standard deviation of steps among seeds that reached the threshold. Bar opacity encodes the fraction of seeds that reached the threshold by the end of training, and the number above each bar is the count of successful seeds. Lower bars and higher opacities indicate faster and more reliable learning.

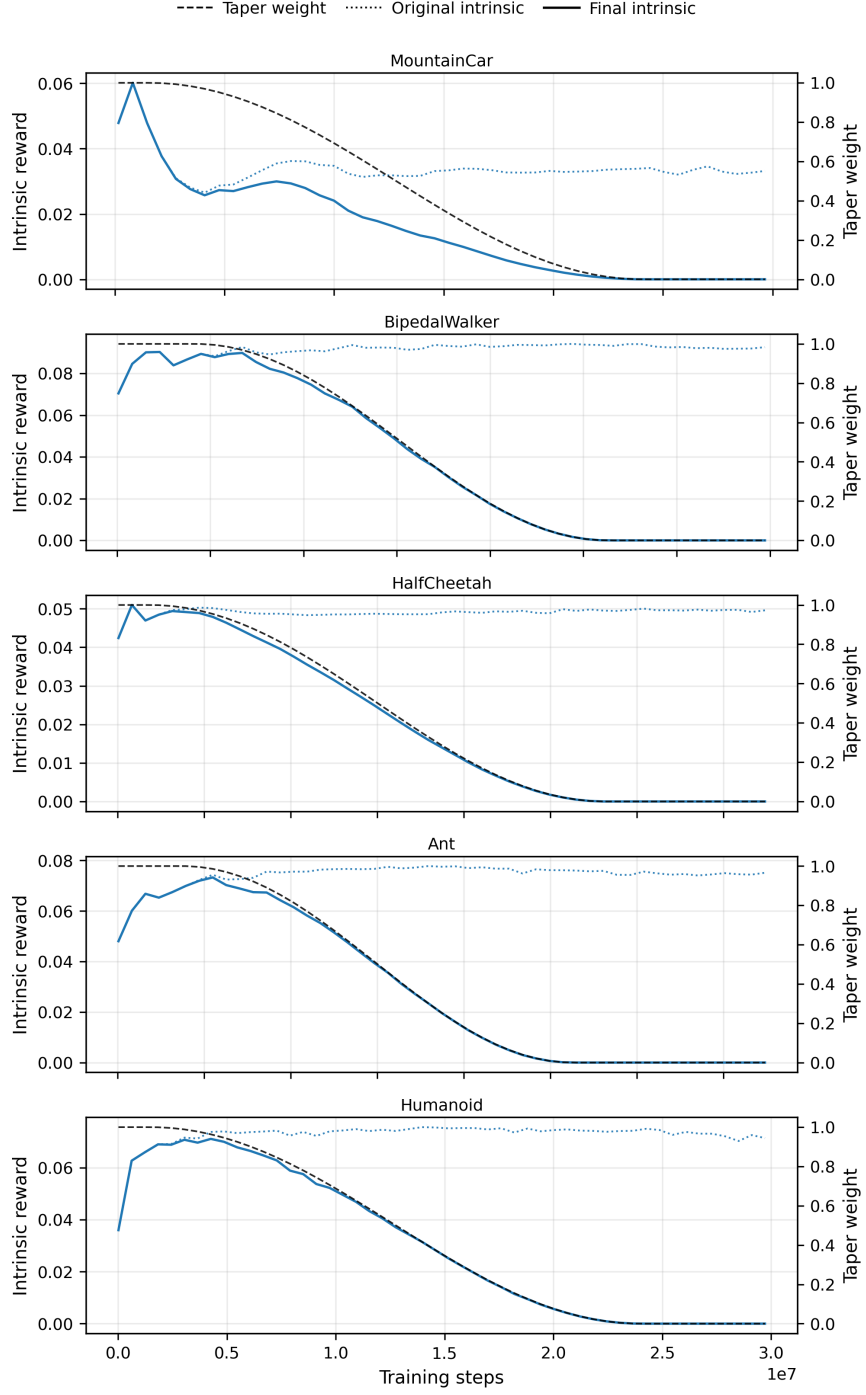


Figure 4. Cosine taper schedule used to anneal intrinsic shaping in the GLPE family (Section 3). The dashed curve (right axis) is the taper weight applied over training progress. The solid curve (left axis) is the batch-mean intrinsic reward actually added to PPO after scaling and clipping. The dotted curve divides the added intrinsic term by the taper weight, showing that the intrinsic signal remains present while the applied contribution vanishes due to annealing.

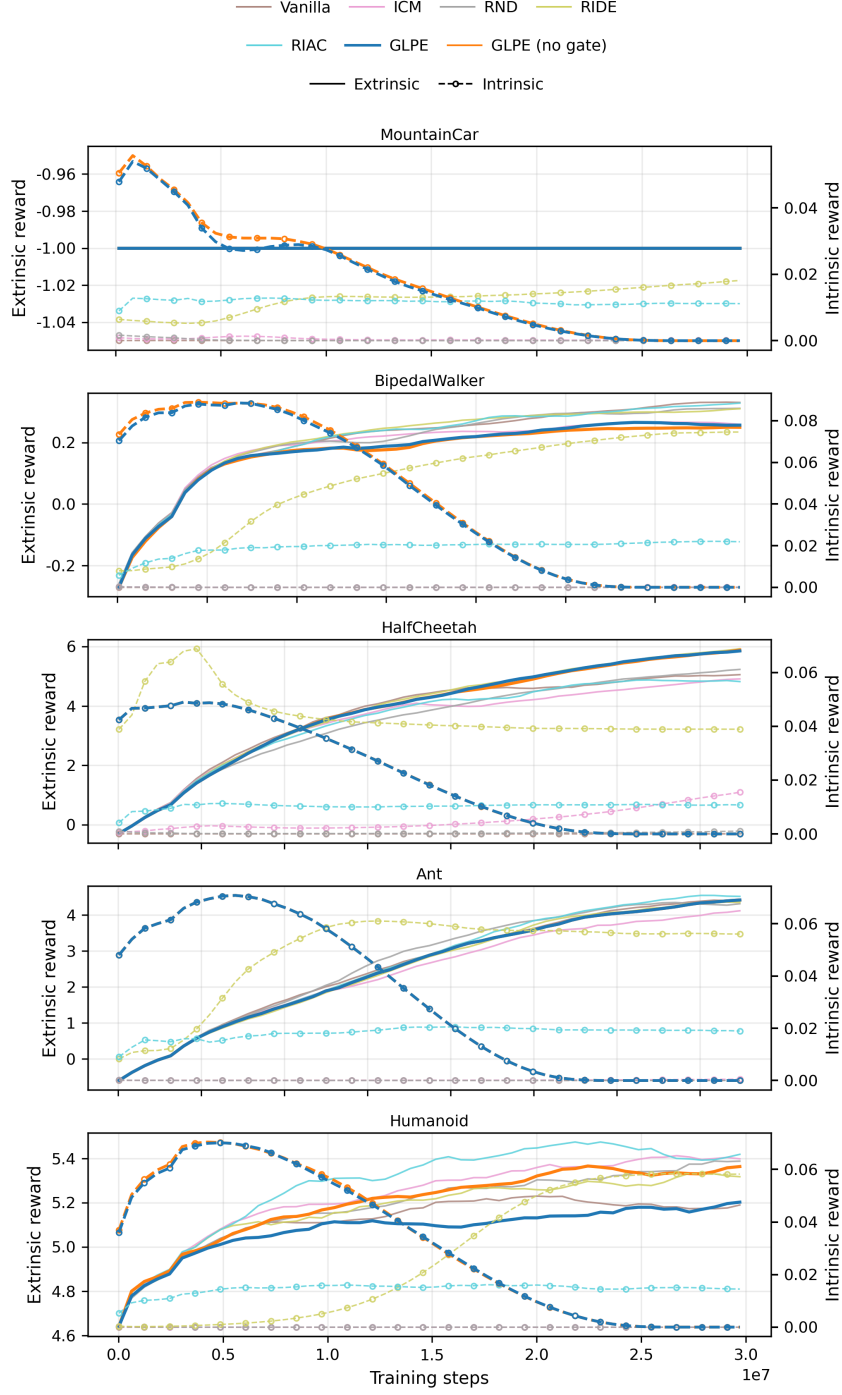


Figure 5. Training-time decomposition of rollout reward. Solid lines show the mean extrinsic reward per environment step collected during PPO rollouts. Dashed lines show the intrinsic shaping component actually added during training, computed as the difference between the logged total reward and the extrinsic reward (after each method’s scaling and clipping; GLPE additionally applies tapering). Intrinsic values use the right axis and extrinsic values use the left axis. This plot is a diagnostic for the optimization signal and is complementary to the evaluation curves, which report extrinsic return without intrinsic shaping.

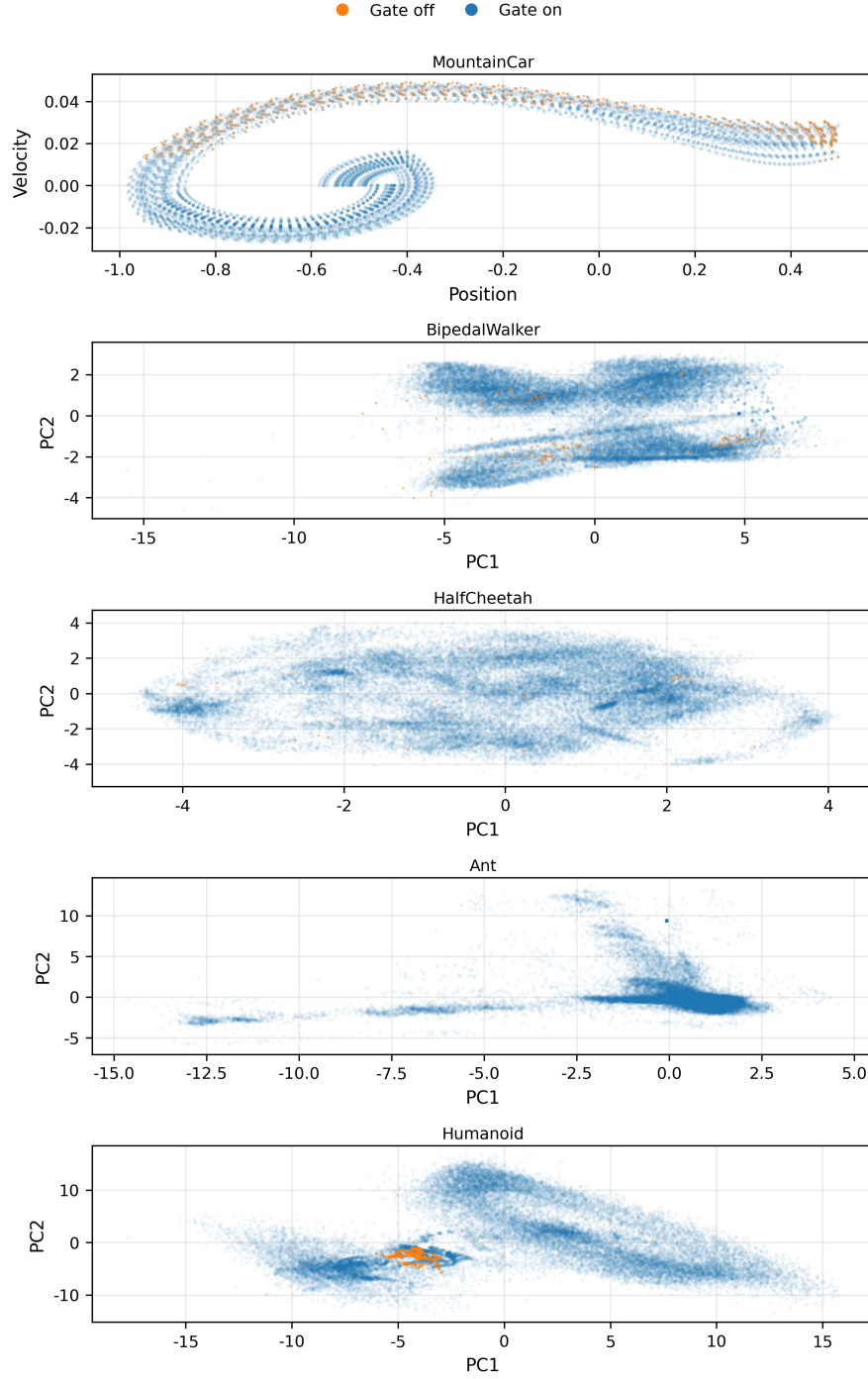


Figure 6. State-space view of GLPE’s gating decisions. Points are states visited in trajectories from the final checkpoint (subsampled); for MOUNTAINCAR-V0 we plot raw position and velocity, and for the remaining environments we project z-scored observations onto the first two principal components. Blue points indicate timesteps where the intrinsic term is active (gate on), while orange points indicate where the gate disables intrinsic shaping (gate off). Across tasks, gate-off points occupy a small and typically localized subset of the visited distribution, indicating that gating suppresses intrinsic shaping only in narrow regions rather than globally.

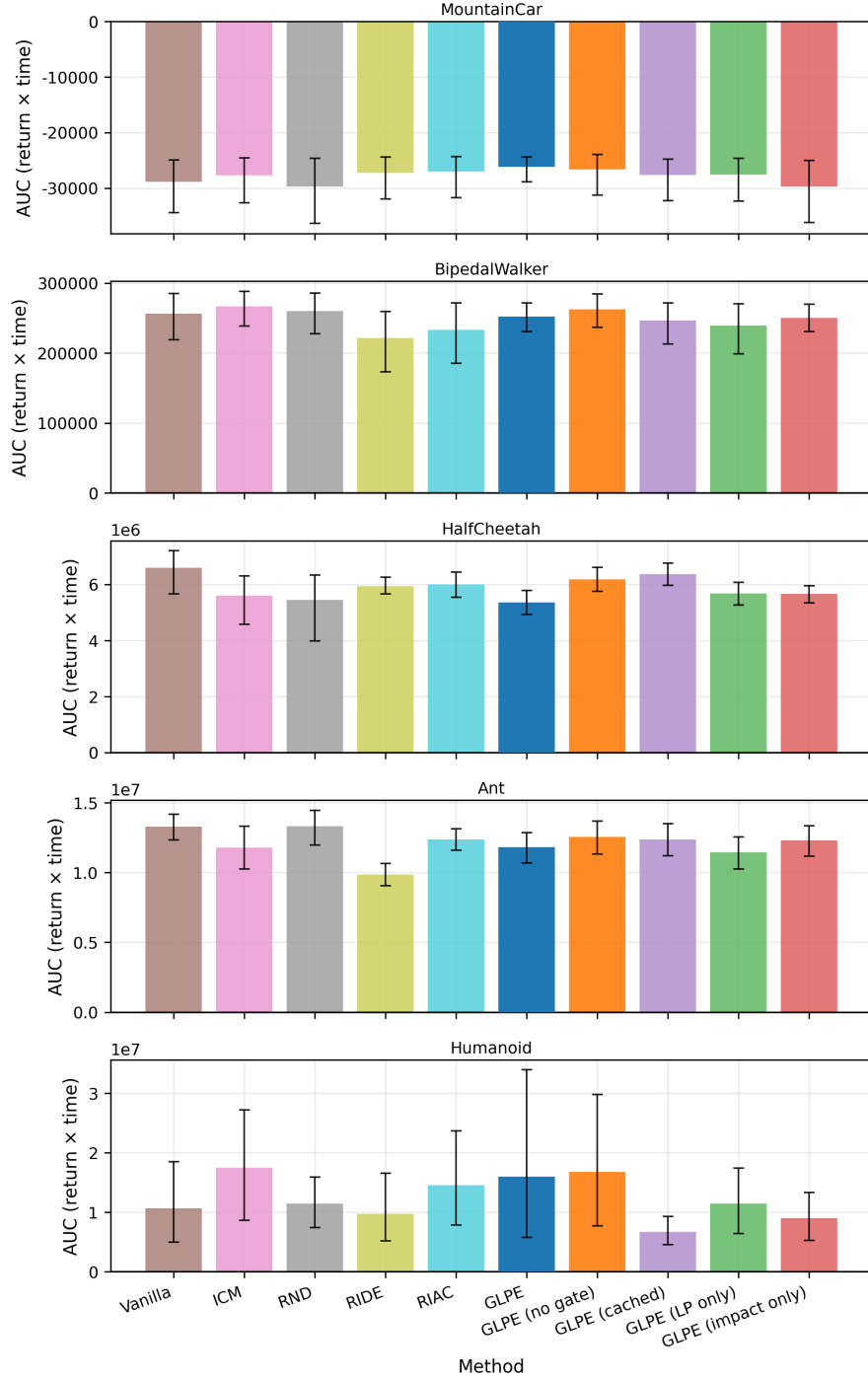


Figure 7. Wall-clock AUC of evaluation performance. For each task, bars report trapezoidal area-under-curve (AUC) of the mean deterministic evaluation return as a function of cumulative training time (Section 4.5). To compare methods fairly when overhead differs, each task uses a common time horizon equal to the minimum final training time among the compared methods, and curves are truncated to this horizon (no extrapolation). Higher is better; for tasks with negative returns (MountainCar), less-negative values yield higher (less-negative) AUC. Error bars are derived by integrating the per-checkpoint 95% confidence bounds on mean return.

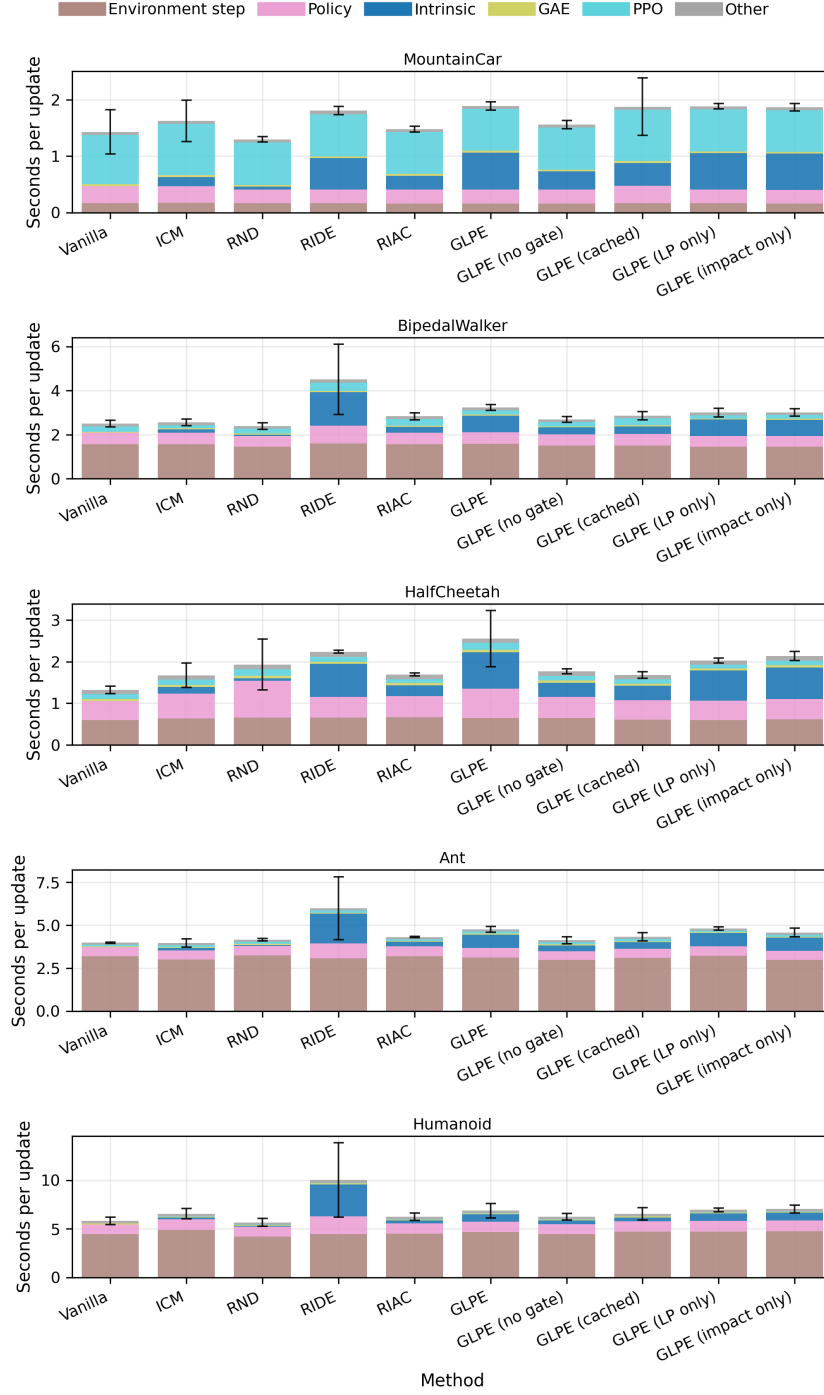


Figure 8. Timing breakdown per PPO update. Each panel shows a stacked decomposition of wall-clock seconds per training update for one task. For each method, we take the final quarter of recorded updates in each run, compute per-component medians, and then average these medians across runs (stacked bars). The black error bar on each stack is a 95% confidence interval for the total seconds per update across runs. This view helps interpret wall-clock tradeoffs: larger intrinsic segments indicate greater exploration-computation overhead, while large environment-step segments indicate simulation-dominated regimes.

References

- Badia, A. P., Sprechmann, P., Vitvitskyi, A., Guo, D., Piot, B., Kapturowski, S., Tieleman, O., Arjovsky, M., Pritzel, A., Bolt, A., and Blundell, C. Never Give Up: Learning Directed Exploration Strategies. *arXiv (Cornell University)*, 2 2020. doi: 10.48550/arxiv.2002.06038. URL <http://arxiv.org/abs/2002.06038>.
- Baranes, A. and Oudeyer, P.-y. R-IAC: robust intrinsically motivated exploration and active learning. *IEEE Transactions on Autonomous Mental Development*, 1(3): 155–169, 10 2009. doi: 10.1109/tamd.2009.2037513. URL <https://doi.org/10.1109/tamd.2009.2037513>.
- Barto, A. G. *Intrinsic Motivation and Reinforcement Learning*. 11 2012. doi: 10.1007/978-3-642-32375-1_{-}2. URL https://doi.org/10.1007/978-3-642-32375-1_2.
- Bellemare, M. G., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., and Munos, R. Unifying Count-Based exploration and intrinsic motivation. *arXiv (Cornell University)*, 6 2016. doi: 10.48550/arxiv.1606.01868. URL <http://arxiv.org/abs/1606.01868>.
- Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., and Efros, A. A. Large-Scale study of Curiosity-Driven Learning. *arXiv (Cornell University)*, 8 2018a. doi: 10.48550/arxiv.1808.04355. URL <http://arxiv.org/abs/1808.04355>.
- Burda, Y., Edwards, H., Storkey, A., and Klimov, O. Exploration by random network distillation. *arXiv (Cornell University)*, 10 2018b. doi: 10.48550/arxiv.1810.12894. URL <http://arxiv.org/abs/1810.12894>.
- Houthoofd, R., Chen, X., Duan, Y., Schulman, J., De Turck, F., and Abbeel, P. VIME: Variational Information Maximizing Exploration. *arXiv (Cornell University)*, 5 2016. doi: 10.48550/arxiv.1605.09674. URL <http://arxiv.org/abs/1605.09674>.
- Mavor-Parker, A. N., Young, K. A., Barry, C., and Griffin, L. D. How to Stay Curious while Avoiding Noisy TVs using Aleatoric Uncertainty Estimation. *arXiv (Cornell University)*, 2 2021. doi: 10.48550/arxiv.2102.04399. URL <http://arxiv.org/abs/2102.04399>.
- Mohamed, S. and Rezende, D. J. Variational information maximisation for intrinsically motivated reinforcement learning. *arXiv (Cornell University)*, 9 2015. doi: 10.48550/arxiv.1509.08731. URL <http://arxiv.org/abs/1509.08731>.
- Ng, A., Harada, D., and Russell, S. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. pp. 278–287, San Francisco, us, 1999. Morgan Kaufmann. doi: 10.5555/645528.657613. URL <https://dl.acm.org/doi/10.5555/645528.657613>.
- Ostrovski, G., Bellemare, M. G., Van Den Oord, A., and Munos, R. Count-Based Exploration with Neural Density Models. *arXiv (Cornell University)*, 3 2017. doi: 10.48550/arxiv.1703.01310. URL <http://arxiv.org/abs/1703.01310>.
- Oudeyer, P.-Y. and Kaplan, F. What is intrinsic motivation? A typology of computational approaches. *Frontiers in Neurorobotics*, 1:6, 1 2007. doi: 10.3389/neuro.12.006.2007. URL <https://doi.org/10.3389/neuro.12.006.2007>.
- Oudeyer, P.-Y., Kaplan, F., and Hafner, V. V. Intrinsic motivation systems for autonomous mental development. *IEEE Transactions on Evolutionary Computation*, 11 (2):265–286, 4 2007. doi: 10.1109/tevc.2006.890271. URL <https://doi.org/10.1109/tevc.2006.890271>.
- Pathak, D., Agrawal, P., Efros, A., and Darrell, T. Curiosity-driven Exploration by Self-supervised Prediction. *arXiv*, 2017. doi: 10.48550/arXiv.1705.05363. URL <https://arxiv.org/abs/1705.05363>.
- Pathak, D., Gandhi, D., and Gupta, A. Self-Supervised Exploration via disagreement. *arXiv (Cornell University)*, 6 2019. doi: 10.48550/arxiv.1906.04161. URL <http://arxiv.org/abs/1906.04161>.
- Raileanu, R. and Rocktäschel, T. RIDE: Rewarding Impact-Driven Exploration for Procedurally-Generated Environments. *arXiv (Cornell University)*, 2 2020. doi: 10.48550/arxiv.2002.12292. URL <http://arxiv.org/abs/2002.12292>.
- Savinov, N., Raichuk, A., Marinier, R., Vincent, D., Pollefeys, M., Lillicrap, T., and Gelly, S. Episodic Curiosity through Reachability. *arXiv (Cornell University)*, 10 2018. doi: 10.48550/arxiv.1810.02274. URL <http://arxiv.org/abs/1810.02274>.
- Schmidhuber, J. *A possibility for implementing curiosity and boredom in Model-Building neural controllers*. 2 1991. doi: 10.7551/mitpress/3115.003.0030. URL <https://doi.org/10.7551/mitpress/3115.003.0030>.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-Dimensional continuous control using generalized advantage estimation. *arXiv (Cornell University)*, 6 2015. doi: 10.48550/arxiv.1506.02438. URL <http://arxiv.org/abs/1506.02438>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. *arXiv*, 2017. doi: 10.48550/arXiv.1707.06347. URL <https://arxiv.org/abs/1707.06347>.
- Singh, S., Barto, A., and Chentanez, N. Intrinsically motivated reinforcement learning. Cambridge, us, 2004. MIT Press.
- Stadie, B. C., Levine, S., and Abbeel, P. Incentivizing exploration in reinforcement learning with deep predictive models. *arXiv (Cornell University)*, 7 2015. doi: 10.48550/arxiv.1507.00814. URL <http://arxiv.org/abs/1507.00814>.

Tang, H., Houthoofd, R., Foote, D., Stooke, A., Chen, X.,
Duan, Y., Schulman, J., De Turck, F., and Abbeel, P.
#Exploration: A study of Count-Based exploration for

deep reinforcement learning. *arXiv (Cornell University)*,
30:2750–2759, 11 2016. doi: 10.48550/arxiv.1611.04717.
URL <http://arxiv.org/abs/1611.04717>.